Architecture Design

Version: 1.0

Status: Draft

Phase: 2 — Technical Design

Purpose

This document defines the high-level technical architecture of the Kommo ↔ Aloware integration.

Its purpose is to describe how the participating systems interact, the architectural principles that guide the solution, and the responsibilities of each component before detailing individual synchronization flows or implementation specifics.

This document intentionally avoids implementation details such as API requests, field mappings, Make scenarios, or retry logic, which are documented separately.

Design Objectives

The architecture has been designed to achieve the following objectives:

- Maintain a clear separation of responsibilities between platforms.
- Synchronize business data with minimal manual intervention.
- Minimize coupling between systems.
- Support independent evolution of each platform.
- Ensure scalability for future automation flows.
- Improve resiliency through event-driven processing.
- Preserve data ownership across systems.
- Facilitate monitoring, troubleshooting, and maintenance.

Architectural Style

The solution follows a combination of architectural patterns.

Event-Driven Architecture

Business events generated by either platform initiate synchronization processes.

Examples include:

- Lead stage changes
- Call completion
- SMS events
- Contact updates
- AI Summary generation

The architecture reacts to events instead of relying on continuous polling whenever webhook support is available.

Orchestration Pattern

Make acts as the orchestration layer between systems.

Rather than allowing direct communication between Kommo and Aloware, every synchronization passes through Make, where business logic is centralized.

This approach provides:

- Centralized workflow management
- Data transformation
- Validation
- Error handling
- Logging
- Monitoring
- Future extensibility

API-Based Integration

Data exchange between systems occurs through REST APIs.

Communication is stateless and request-based.

Each synchronization is executed independently using authenticated API requests.

Loose Coupling

Neither Kommo nor Aloware contains business logic specific to the other platform.

Platform-specific behavior remains isolated inside the integration layer.

This reduces dependencies and simplifies future platform changes.

System Components

Kommo CRM

Primary responsibilities:

- Customer relationship management
- Lead lifecycle management
- Sales pipeline
- Contacts
- Companies
- Sales ownership

Kommo is the authoritative source for commercial information.

Aloware

Primary responsibilities:

- Outbound calling
- Inbound calling
- SMS
- Call recordings
- AI Call Summary
- Communication history

Aloware is the authoritative source for communication data.

Make

Primary responsibilities:

- Workflow orchestration
- Event processing
- API communication
- Data transformation

- Validation
- Error management
- Retry execution
- Logging
- Monitoring integration processes

No business data is permanently owned by Make.

High-Level Interaction Model

The integration is organized around independent synchronization flows.

Each flow follows the same logical lifecycle:

1. Business event occurs.
2. Event is received by Make.
3. Event is validated.
4. Required data is retrieved.
5. Data is transformed.
6. Target API is executed.
7. Result is validated.
8. Logs are generated.
9. Process finishes.

Each synchronization is independent from the others.

Communication Model

The solution combines two communication mechanisms.

Event Reception

Preferred mechanism:

- Webhooks

Alternative mechanism:

- Scheduled polling (only if required by platform limitations)

Data Exchange

Communication between systems is performed through authenticated REST APIs.

No direct database access exists between platforms.

Synchronization Principles

The architecture follows these synchronization principles.

System of Record

Each business entity has a single authoritative owner.

Commercial information belongs to Kommo.

Communication information belongs to Aloware.

Ownership is never shared.

Eventual Consistency

The systems are not expected to remain synchronized in real time.

Synchronization occurs after relevant events are processed successfully.

Short synchronization delays are considered acceptable.

Independent Processing

Each synchronization executes independently.

A failure in one workflow must not interrupt unrelated synchronization processes.

Stateless Execution

Each workflow execution contains all information required to complete its operation.

No execution depends on previous runtime state stored within Make.

Idempotent Design

Whenever possible, synchronization operations should be repeatable without producing duplicate business effects.

This minimizes the impact of retries and duplicate event delivery.

Architectural Principles

The technical design is guided by the following principles.

Separation of Responsibilities

Each platform performs only the functions that belong to its domain.

Single Source of Truth

Each data domain has a single owner.

Explicit Data Flow

Every data movement between systems should be intentional, documented, and traceable.

Centralized Integration Logic

Business rules specific to synchronization remain inside the integration layer rather than inside either platform.

Failure Isolation

Errors should affect only the current synchronization process.

Other workflows must continue operating normally.

Observability

Every synchronization should generate sufficient operational information to support diagnostics and auditing.

Extensibility

The architecture should allow future synchronization flows to be incorporated without requiring structural redesign.

Architectural Constraints

The design assumes the following constraints.

- Communication occurs exclusively through supported APIs and Webhooks.
- No direct database integration exists.
- Platform permissions define accessible operations.
- API rate limits must be respected.
- Authentication mechanisms remain platform-specific.
- Make serves as the only orchestration layer.

Out of Scope

The following topics are intentionally excluded from this document:

- Detailed Make scenario design
- Field mappings
- Pipeline mappings
- Agent mappings
- API endpoint specifications
- Retry implementation
- Error handling procedures
- Logging implementation
- Monitoring implementation
- Security implementation details

These topics are documented independently within the Technical Design phase.

Related Documents

This document is complemented by:

- Integration Flows Design
- Synchronization Model
- Field Mapping
- Stage Mapping
- Agent Mapping
- Data Transformation Rules
- Validation Rules
- Error Handling Strategy
- Retry Strategy
- Logging Strategy
- Monitoring Strategy
- Security Considerations
- Technical Design Summary